Icône Ajouter Ajouter une couverture

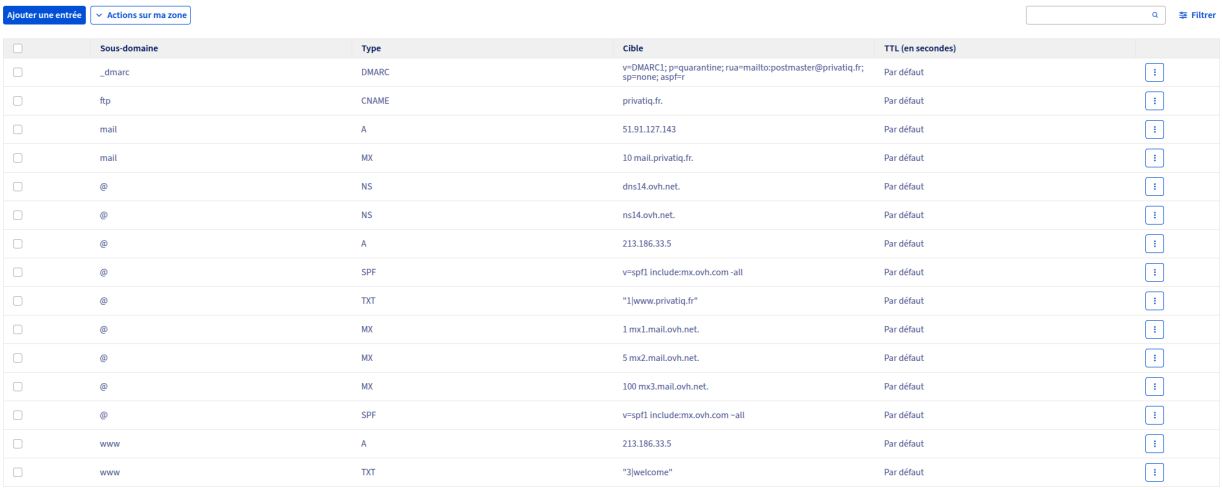
3. Installation Mail

Commandes

```shell
cd /opt
sudo git clone https://github.com/mailcow/mailcow-dockerized
cd mailcow-dockerized/
sudo ./generate_config.sh
FQDN : mail.privatiq.fr
Timezone [Etc/UTC]: UTC
Branch with it's number [1/2/3] : 1
Create it with IPv6 settings? [Y/n] Y
sudo docker compose pull
sudo docker compose up -d
```

Changement des entrée DNS

Avant :



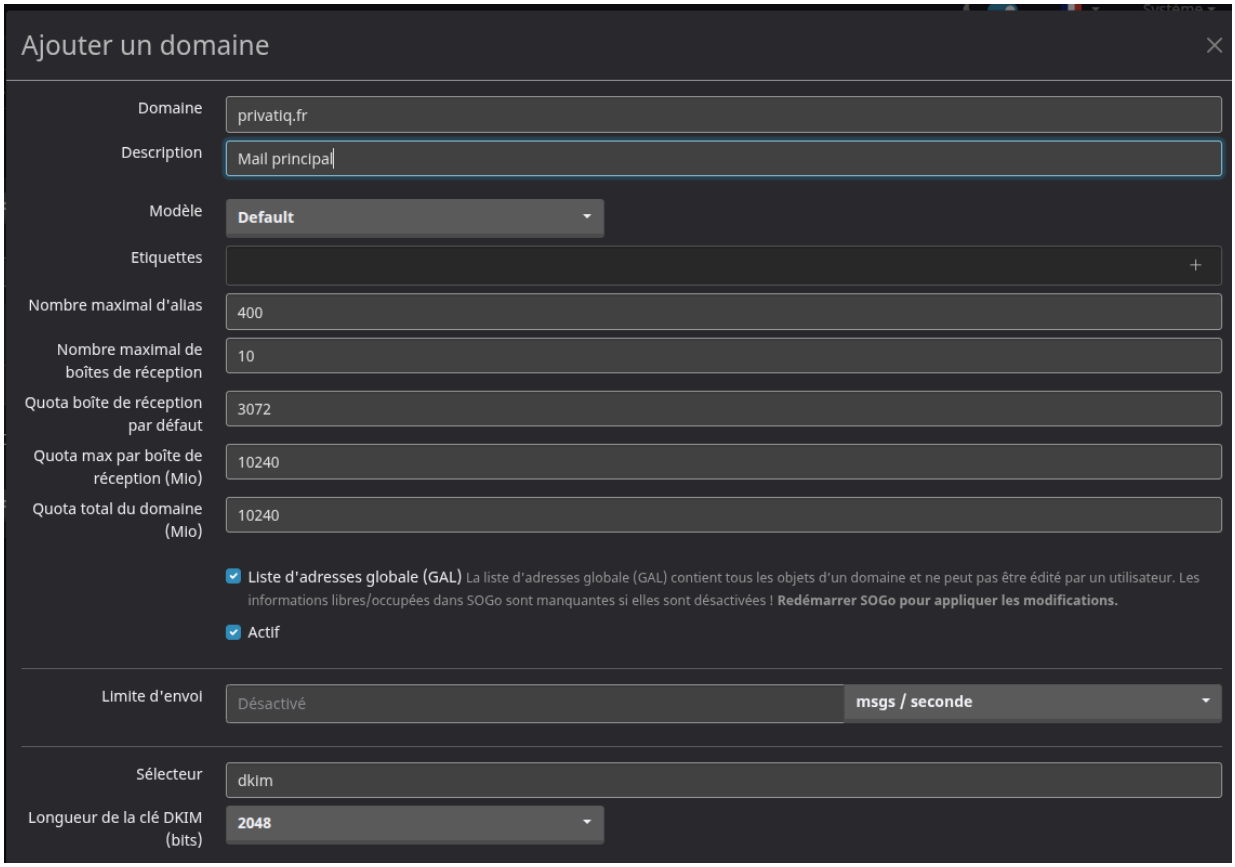
Ajouter une entrée | Actions sur ma zone | Filtrer

Sous-domaine	Type	Cible	TTL (en secondes)
_dmarc	DMARC	v=DMARC1; p=quarantine; rua=mailto:postmaster@privatiq.fr; sp=none; aspf=r	Par défaut
ftp	CNAME	privatiq.fr.	Par défaut
mail	A	51.91.127.143	Par défaut
mail	MX	10 mail.privatiq.fr.	Par défaut
@	NS	dns14.ovh.net.	Par défaut
@	NS	ns14.ovh.net.	Par défaut
@	A	213.186.33.5	Par défaut
@	SPF	v=spf1 include:mx.ovh.com -all	Par défaut
@	TXT	"1|www.privatiq.fr"	Par défaut
@	MX	1 mx1.mail.ovh.net.	Par défaut
@	MX	5 mx2.mail.ovh.net.	Par défaut
@	MX	100 mx3.mail.ovh.net.	Par défaut
@	SPF	v=spf1 include:mx.ovh.com -all	Par défaut
www	A	213.186.33.5	Par défaut
www	TXT	"3|welcome"	Par défaut

Accès à MailCow avec

https://mail.privatiq.fr

Ajout du domaine :



Une fois le domaine crée, on peut ajouter le DKIM dans Système > Configuration > Option

Ajout du DKIM dans les entrée DNS de OVH

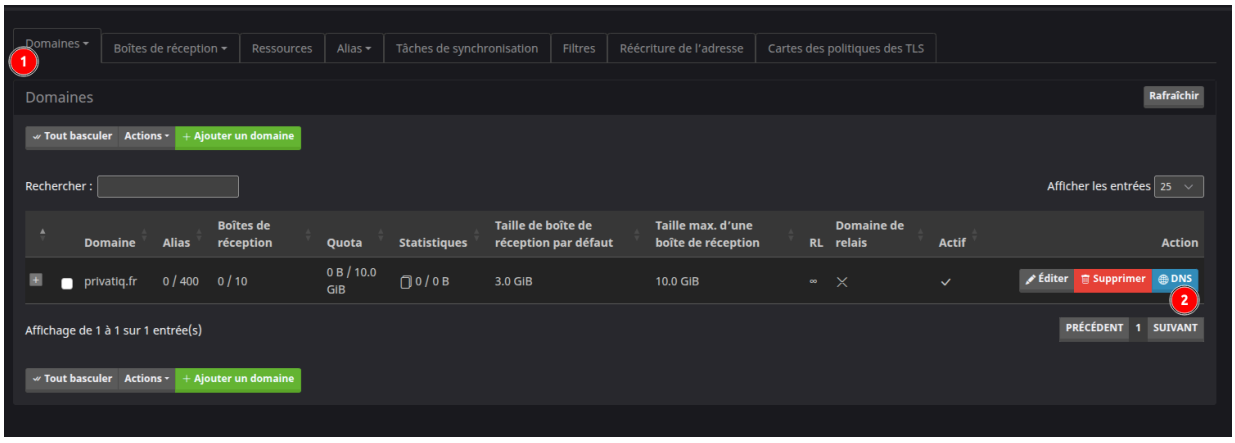
dkim._domainkey	DKIM	v=DKIM1; h=sha256; k=rsa; p=MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAp5Y3E s=email;	Par défaut

Modification du SPF

- `mx` ✅ autorise ton serveur mail
- `a` ✅ autorise ton domaine
- `ip4` ✅ bétonne la config
- évite les faux positifs spam

@	SPF	v=spf1 ip4:51.91.127.143 mx a -all	Par défaut

Vérification des DNS validé par mailcow



Fix des reverse DNS sur OVH

Avant :

```
51.91.127.143 → vps-9319a541.vps.ovh.net
```

Après :

```
51.91.127.143 → mail.privatiq.fr
```

Modification du mailcow.conf

AVANT

```
SMTP_PORT=25
SMTPS_PORT=465
SUBMISSION_PORT=587
IMAP_PORT=143
IMAPS_PORT=993
POP_PORT=110
POPS_PORT=995
SIEVE_PORT=4190
DOVEADM_PORT=127.0.0.1:19991
SQL_PORT=127.0.0.1:13306
REDIS_PORT=127.0.0.1:7654
```

APRES

```
SMTP_PORT=25
SMTPS_PORT=465
SUBMISSION_PORT=587
IMAP_PORT=
IMAPS_PORT=993
POP_PORT=
POPS_PORT=
SIEVE_PORT=
DOVEADM_PORT=127.0.0.1:19991
SQL_PORT=127.0.0.1:13306
REDIS_PORT=127.0.0.1:7654
```

Problème identifié : UFW était actif mais inefficace contre Docker.

Explication

- UFW filtre les chaînes `INPUT`
- Docker utilise `iptables` (chaîne `FORWARD`)
- Docker contourne donc les règles UFW

👉 Résultat : les ports restaient accessibles malgré les règles de blocage

---

✅ Solution mise en place

Utilisation de la chaîne `DOCKER-USER` pour filtrer le trafic AVANT Docker.

Règles appliquées

```shell
sudo iptables -I DOCKER-USER -p tcp --dport 110 -j DROP
sudo iptables -I DOCKER-USER -p tcp --dport 143 -j DROP
sudo iptables -I DOCKER-USER -p tcp --dport 995 -j DROP
sudo iptables -I DOCKER-USER -p tcp --dport 4190 -j DROP
sudo iptables -I DOCKER-USER -p tcp --dport 465 -j DROP
```

✅ Vérification

1. Vérification des règles

```shell
sudo iptables -L DOCKER-USER -n -v
```

Les règles DROP doivent apparaître en haut de la chaîne.

En plus, j'ai comenté les ligne qui permette l'écoute sur les ports non nécessaire :

nano docker-compose.yml :

```shell
      ports:
        - "${DOVEADM_PORT:-127.0.0.1:19991}:12345"
#        - "${IMAP_PORT:-143}:143"
        - "${IMAPS_PORT:-993}:993"
#        - "${POP_PORT:-110}:110"
#        - "${POPS_PORT:-995}:995"
#        - "${SIEVE_PORT:-4190}:4190"
```

Nous avons plus que les ports utiles :

```shell
Netid                State                  Recv-Q                  Send-Q
Local Address:Port                                  Peer Address:Port
Process
udp                  UNCONN                 0                       0
127.0.0.54:53                                       0.0.0.0:*
udp                  UNCONN                 0                       0
127.0.0.53%lo:53                                       0.0.0.0:*
udp                  UNCONN                 0                       0
51.91.127.143%ens3:68                                          0.0.0.0:*
tcp                  LISTEN                 0                       4096
127.0.0.1:13306                                     0.0.0.0:*
tcp                  LISTEN                 0                       4096
127.0.0.1:7654                                      0.0.0.0:*
tcp                  LISTEN                 0                       4096
127.0.0.53%lo:53                                       0.0.0.0:*
```

```
tcp        LISTEN     0          128        0.0.0.0:51478      0.0.0.0:*
tcp        LISTEN     0          4096       0.0.0.0:993        0.0.0.0:*
tcp        LISTEN     0          4096       0.0.0.0:587        0.0.0.0:*
tcp        LISTEN     0          4096       0.0.0.0:465        0.0.0.0:*
tcp        LISTEN     0          4096       0.0.0.0:443        0.0.0.0:*
tcp        LISTEN     0          4096       127.0.0.54:53         0.0.0.0:*
tcp        LISTEN     0          4096       127.0.0.1:19991       0.0.0.0:*
tcp        LISTEN     0          4096       0.0.0.0:80         0.0.0.0:*
tcp        LISTEN     0          4096       0.0.0.0:25         0.0.0.0:*
tcp        LISTEN     0          128        [::]:51478         [::]:*
tcp        LISTEN     0          4096       [::]:993           [::]:*
tcp        LISTEN     0          4096       [::]:587           [::]:*
tcp        LISTEN     0          4096       [::]:465           [::]:*
tcp        LISTEN     0          4096       [::]:443           [::]:*
tcp        LISTEN     0          4096       [::]:80            [::]:*
tcp        LISTEN     0          4096       [::]:25            [::]:*
```

Modifier l'autoconfiguration Mailcow pour SMTP port 587 (STARTTLS)

Par défaut, Mailcow liste le port 465 (SSL) en premier dans son fichier d'autoconfig. Voici comment mettre le port 587 (STARTTLS) en priorité.

Étape 1 — Modifier le fichier autoconfig.php

Shell

```
sudo nano /opt/mailcow-dockerized/data/web/autoconfig.php
```

Trouve les deux blocs `outgoingServer` et inverse leur ordre pour que STARTTLS/587 soit en premier :

XML

```
<outgoingServer type="smtp">
   <hostname><?=$autodiscover_config['smtp']['server']; ?></hostname>
   <port><?=$autodiscover_config['smtp']['tlsport']; ?></port>
   <socketType>STARTTLS</socketType>
   <username>%EMAILADDRESS%</username>
   <authentication>password-cleartext</authentication>
</outgoingServer>
<outgoingServer type="smtp">
   <hostname><?=$autodiscover_config['smtp']['server']; ?></hostname>
   <port><?=$autodiscover_config['smtp']['port']; ?></port>
   <socketType>SSL</socketType>
   <username>%EMAILADDRESS%</username>
```

```
    <authentication>password-cleartext</authentication>
</outgoingServer>
```

Étape 2 — Appliquer la modification dans le conteneur php-fpm

Le fichier sur l'hôte ne suffit pas, il faut le copier dans le conteneur :

Shell

```
sudo docker cp /opt/mailcow-dockerized/data/web/autoconfig.php \
  mailcowdockerized-php-fpm-mailcow-1:/web/autoconfig.php
```

Puis redémarre php-fpm pour vider le cache opcache :

Shell

```
sudo docker compose -f /opt/mailcow-dockerized/docker-compose.yml restart php-fpm-
mailcow
```

Étape 3 — Vérifier

Shell

```
curl https://autoconfig.privatiq.fr/mail/config-v1.1.xml | grep -A4 "outgoingServer"
```

Le résultat doit montrer **587 / STARTTLS en premier** :

XML

```
<outgoingServer type="smtp">
   <hostname>mail.privatiq.fr</hostname>
   <port>587</port>
   <socketType>STARTTLS</socketType>
   ...
```

⚠️ Après une mise à jour Mailcow

La modification dans le conteneur est perdue après un `docker compose pull`. Réapplique simplement l'étape 2 après chaque mise à jour :

Shell

```
sudo docker cp /opt/mailcow-dockerized/data/web/autoconfig.php \
  mailcowdockerized-php-fpm-mailcow-1:/web/autoconfig.php

sudo docker compose -f /opt/mailcow-dockerized/docker-compose.yml restart php-fpm-mailcow
```